Rede Convolucional 1D from Scratch

Bernardo Mendes Rebello

Lucas Pinto Avelar

O que foi implementado

- Camada convolucional
- Camada totalmente conectada
- Max Pooling
- Flatten
- Otimizador
- Backpropagation
- Funções de ativação

Pegamos pronto

- Função de perda (Binary cross entropy)

Implementação

Todas as camadas e componentes definidos em classes do python

- Implementação facilitada
  - Backpropagation
  - Forward
- Utilização parecida com o pytroch: Sequential
- Camadas e componente configuráveis
- Leitura facilitada

- Queda de desempenho

```python
class Sequential:
    def __init__(self, layers:list):
        self.layers = layers

    def forward(self, X):
        for layer in self.layers:
            X = layer.forward(X)
        return X
    def backward(self, grad):
        for layer in reversed(self.layers):
            grad = layer.backward(grad)[0] #pegar só o dX
```

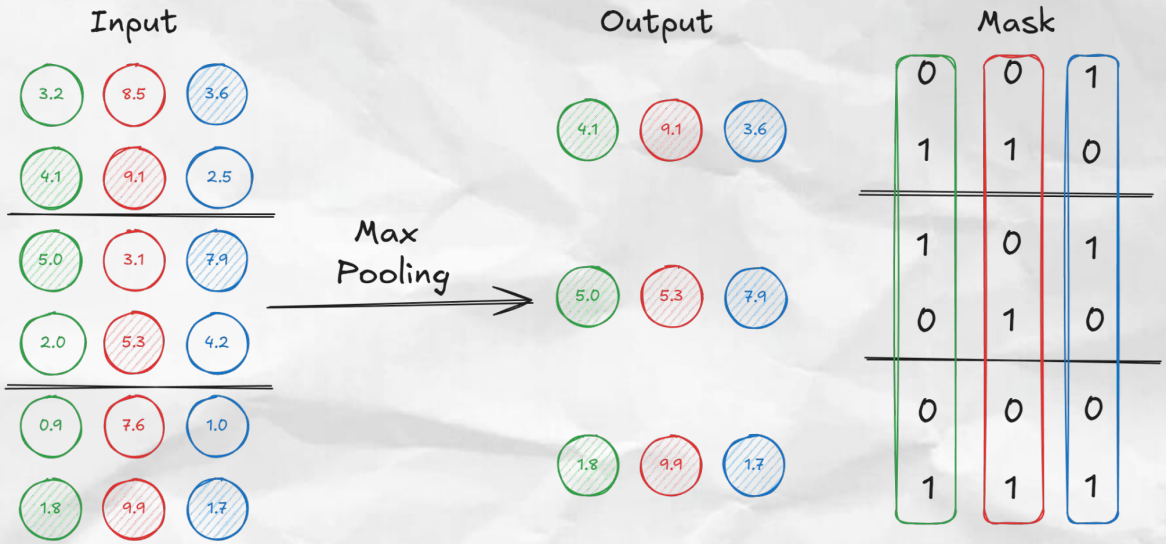
Funções de Ativação e Entropia Cruzada

```python
class ReLU:
    def __init__(self):
        self.saved_input = None

    def forward(self, input_data: np.ndarray) -> np.ndarray:
        self.saved_input = input_data
        return np.maximum(0, input_data)

    def backward(self, erro: np.ndarray) -> np.ndarray:
        # O gradiente será a derivada da ReLU (1 para x > 0, 0 para x <= 0) * erro
        dX = erro.copy()
        dX[self.saved_input <= 0] = 0
        return dX

class Sigmoid:
    def __init__(self):
        self.saved_input = None

    def forward(self, input_data: np.ndarray) ->np.ndarray:
        self.saved_input = input_data
        return self.sigmoid(input_data)

    def sigmoid(self, x: np.ndarray) -> np.ndarray:
        return 1/(1 + np.exp(-x))


    def backward(self, erro: np.ndarray) -> np.ndarray:
        sigmoid_output = self.sigmoid(self.saved_input)
        dX = erro * ( sigmoid_output * (1 - sigmoid_output) )
        return dX
```

```python
class BinaryCrossEntropy:
    def forward(self, y_pred, y_true):
        # y_pred: Saída da rede (após Sigmoid), valores entre 0 e 1
        # y_true: Rótulos reais (0 ou 1)

        epsilon = 1e-15 #evitar log(0) que daria -infinito e quebraria a rede
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)

        self.y_pred = y_pred
        self.y_true = y_true

        # Fórmula da Entropia Cruzada Binária
        loss = - (y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

        return np.mean(loss)

    def backward(self):
        # Derivada da Loss em relação à previsão (y_pred)
        # dL/dy_pred = -(y/y_pred) + (1-y)/(1-y_pred)

        epsilon = 1e-15
        y_pred = np.clip(self.y_pred, epsilon, 1 - epsilon)

        grad_input = - (self.y_true / y_pred) + ((1 - self.y_true) / (1 - y_pred))

        return grad_input / y_pred.size
```

Max Pooling

Forward Max Pooling

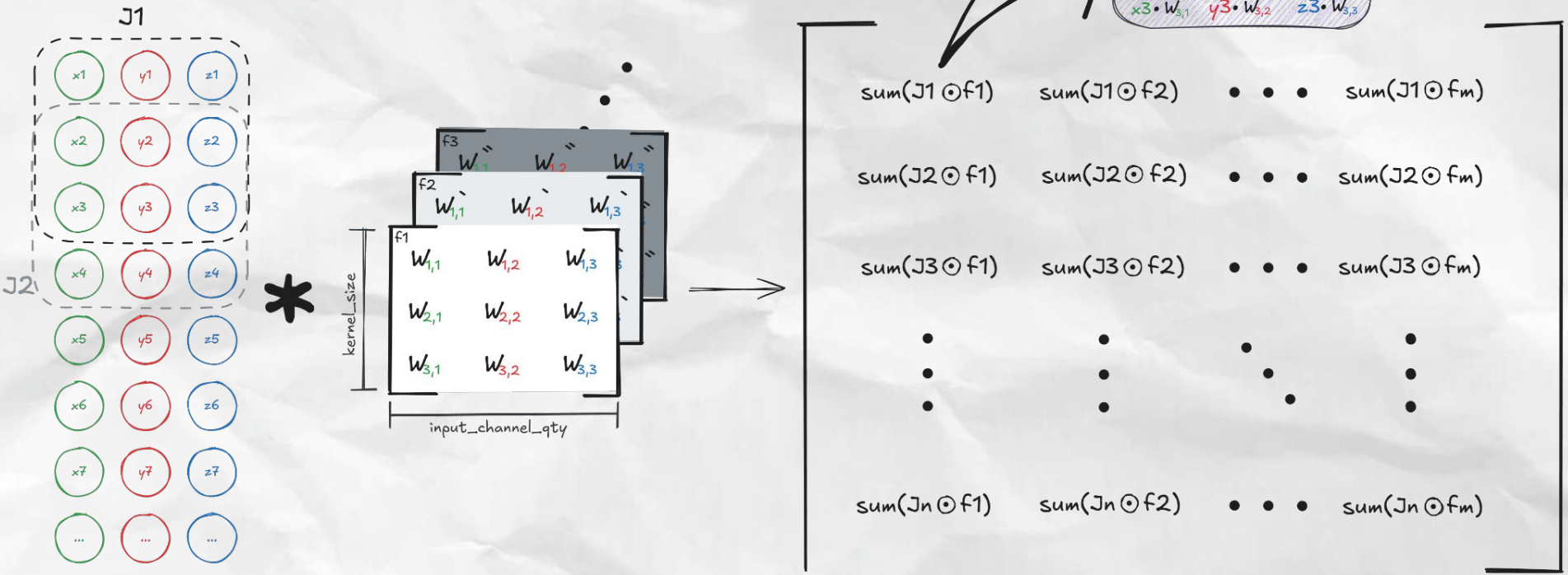



```python
class MaxPooling:
    def __init__(self, size:int):
        self.size = size # tamanho da janela do max pooling
        self.saved_input = None
        self.mask = None

    def forward(self, input_data):
        self.saved_input = input_data
        result_size = int(np.floor(input_data.shape[0]/self.size))
        result = np.zeros((result_size, input_data.shape[1]))

        self.mask = np.zeros_like(input_data)

        for i in range(result_size):
            pos = i*self.size
            janela = input_data[pos:pos+self.size]
            result[i] = np.max(janela, axis=0)

            #pegar os indices dos maximos
            idx_max = janela.argmax(axis=0)
            #colocar 1 na posicao dos maximos na mask
            self.mask[pos + idx_max, np.arange(input_data.shape[1])] = 1

        return result
```

Backward Max Pooling


```python
class MaxPooling:
    def __init__(self, size:int):
        self.size = size # tamanho da janela do max pooling
        self.saved_input = None
        self.mask = None

    def forward(self, input_data):
        ...
        return result

    def backward(self, d_output):
        d_output = np.repeat(d_output, self.size, axis = 0)

        dX = np.zeros_like(self.saved_input) #do tamanho exato da entrada origianl
        limite_preenchimento = min(d_output.shape[0], dX.shape[0])
        dX[:limite_preenchimento] = d_output[:limite_preenchimento]
        dX = dX*self.mask
        return dX
```

Forward CNN 1D



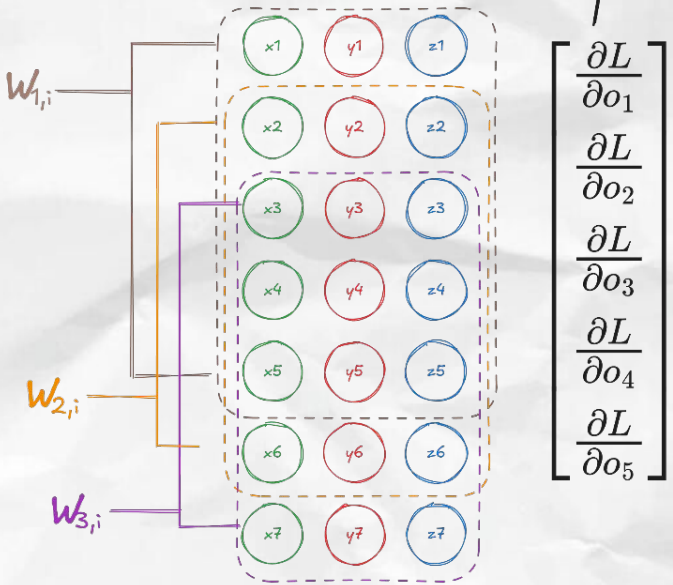
```python
class ConvLayer:
    '''Implementação sem usar matriz de pesos'''
    def __init__(self, kernel_size:int, weights:np.ndarray = None, bias:np.ndarray = None,
in_channels:int = 1, out_channels:int = 1, stride:int = 1, dilation:int = 1,
zero_padding:int = 0):
        if weights is None:
            weights = np.random.randn(out_channels, kernel_size,
in_channels).astype(np.float32)
        else:
            assert weights.shape == (out_channels, kernel_size,in_channels), "Os pesos
devem estar no formato ( in_chanel, kernel_size, out_chanel)"

        if bias is None:
            bias = np.random.randn(out_channels).astype(np.float32)
        else:
            assert bias.shape == (out_channels,), "O bias deve estar no formato (
out_chanel, )"

        self.kernel_size = kernel_size
        self.stride = stride
        self.dilation = dilation
        self.zero_padding = zero_padding

        self.saved_input = None

        self.bias = bias # (um bias para cada filtro)
        self.in_channels = in_channels
        self.out_channels = out_channels #será a quantidade de filtros

        tamanho_linhas_dilatacao = dilation * kernel_size - dilation + 1

                #qtd matrizes            qtd_linhas_filtro   qtdcolunas
        temp = np.zeros((out_channels, tamanho_linhas_dilatacao, in_channels),
dtype=weights.dtype) #cria uma matriz de zeros para ajustar o formato dos pesos
        temp[:,::dilation,:] = weights #preenche o vetor de zeros com os pesos pulando o
valor da dilatação
        self.dilated_weights = temp #pesos com a dilatação aplicada

        self.weights = self.dilated_weights[:, ::self.dilation, :]#pesos originais
```

```python
def forward(self, input_data:np.ndarray) -> np.ndarray:
        assert input_data.shape[1] == self.in_channels

        #aplicar padding em cada canal
        input_used = np.pad(input_data, ((self.zero_padding, self.zero_padding),(0,0)))
        self.saved_input = input_used #salva o input para o backpropagation


        #Calculo do tamanho do resultado
        result_size = int(np.ceil((input_used.shape[0] - self.dilated_weights.shape[1] + 1)
/ self.stride))
        self.result_size = result_size
        result = np.zeros((result_size, self.out_channels))

        #para cada i (linha do resultado)
        for i in range(result_size):
            pos_acesso = i * self.stride #ajusta o acesso
            for j in range(self.out_channels):
                filtro_atual = self.dilated_weights[j]
                            #recorte do input (janela)    #kernel
                janela = input_used[pos_acesso:pos_acesso+filtro_atual.shape[0]]

                element_wise = janela * filtro_atual
                result[i, j] = np.sum(element_wise)

        result += self.bias  # Adiciona o vetor bias
        return result
```

Cálculo dW

$$\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \\ w_{3,1} & w_{3,2} & w_{3,3} \end{bmatrix}$$

$$\frac{\partial L}{\partial W_k} = \sum_i \left(\frac{\partial L}{\partial O_i} \cdot X_{i+k} \right)$$

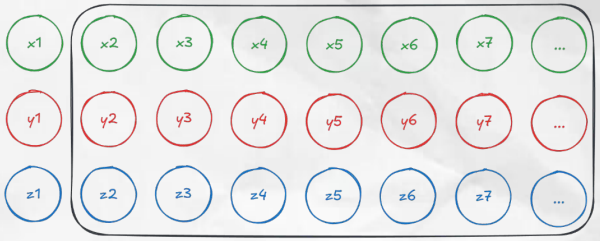


$$\begin{bmatrix} \frac{\partial L}{\partial o_1} \\ \frac{\partial L}{\partial o_2} \\ \frac{\partial L}{\partial o_3} \\ \frac{\partial L}{\partial o_4} \\ \frac{\partial L}{\partial o_5} \end{bmatrix}$$

$$\frac{\partial L}{\partial W} = \begin{bmatrix} \overset{W_{1,1}}{\sum_{i=1}^{5} \frac{\partial L}{\partial o_i} \cdot x_i} & \overset{W_{1,2}}{\sum_{i=1}^{5} \frac{\partial L}{\partial o_i} \cdot y_i} & \overset{W_{1,3}}{\sum_{i=1}^{5} \frac{\partial L}{\partial o_i} \cdot z_i} \\ \overset{W_{2,1}}{\sum_{i=1}^{5} \frac{\partial L}{\partial o_i} \cdot x_{i+1}} & \overset{W_{2,2}}{\sum_{i=1}^{5} \frac{\partial L}{\partial o_i} \cdot y_{i+1}} & \overset{W_{2,3}}{\sum_{i=1}^{5} \frac{\partial L}{\partial o_i} \cdot z_{i+1}} \\ \overset{W_{3,1}}{\sum_{i=1}^{5} \frac{\partial L}{\partial o_i} \cdot x_{i+2}} & \overset{W_{3,2}}{\sum_{i=1}^{5} \frac{\partial L}{\partial o_i} \cdot y_{i+2}} & \overset{W_{3,3}}{\sum_{i=1}^{5} \frac{\partial L}{\partial o_i} \cdot z_{i+2}} \end{bmatrix}$$

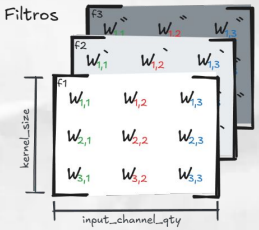
Calcular

2ª linha de
todos os filtros



$$\times \quad \frac{\partial L}{\partial O} = \begin{bmatrix} \frac{\partial L}{\partial O_{1,1}} & \frac{\partial L}{\partial O_{1,2}} & \cdots & \frac{\partial L}{\partial O_{1,m}} \\ \frac{\partial L}{\partial O_{2,1}} & \frac{\partial L}{\partial O_{2,2}} & \cdots & \frac{\partial L}{\partial O_{2,m}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial L}{\partial O_{n,1}} & \frac{\partial L}{\partial O_{n,2}} & \cdots & \frac{\partial L}{\partial O_{n,m}} \end{bmatrix}$$

$$= \begin{bmatrix} \frac{\partial L}{\partial W_{2,1}} & \frac{\partial L}{\partial W'_{2,1}} & \frac{\partial L}{\partial W''_{2,1}} \\ \frac{\partial L}{\partial W_{2,2}} & \frac{\partial L}{\partial W'_{2,2}} & \frac{\partial L}{\partial W''_{2,2}} \\ \frac{\partial L}{\partial W_{2,3}} & \frac{\partial L}{\partial W'_{2,3}} & \frac{\partial L}{\partial W''_{2,3}} \end{bmatrix}$$



$$\frac{\partial L}{\partial O_{1,1}} \begin{bmatrix} x1 \\ y1 \\ z1 \end{bmatrix} + \frac{\partial L}{\partial O_{2,1}} \begin{bmatrix} x2 \\ y2 \\ z2 \end{bmatrix} + \cdots + \frac{\partial L}{\partial O_{n,1}} \begin{bmatrix} xn \\ yn \\ zn \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial W_{2,1}} \\ \frac{\partial L}{\partial W_{2,2}} \\ \frac{\partial L}{\partial W_{2,3}} \end{bmatrix}$$

```python
    def calcular_dW(self, erros):
        #nosso resultado será:
        # dL/dW_{1,1} dL/dW'_{1,1} dL/dW''_{1,1} ...
        # dL/dW_{1,2} dL/dW'_{1,2} dL/dW''_{1,1} ...
        #     ...         ...           ...

        dW = np.zeros((self.kernel_size, self.in_channels,
self.out_channels))
        n_outputs = erros.shape[0]

        for linha_wi in range(self.kernel_size):

            #onde comeca o w_i
            start_index = linha_wi * self.dilation

            # pega todos os inputs que w_i multiplicou
            input_slice = self.saved_input[start_index : : self.stride]

            # Cortamos para garantir o tamanho exato (caso sobre padding no
final)
            input_slice = input_slice[:n_outputs]

            # Verificação de segurança
            if input_slice.shape[0] != n_outputs:
                # Isso pode acontecer se o padding/stride no forward gerou
janelas que o backward não alcança
                # Geralmente resolve-se cortando o erro ou o input para o
menor denominador comum
                min_len = min(input_slice.shape[0], n_outputs)
                input_slice = input_slice[:min_len]
                # erros_ajustado = erros[:min_len]

            dW[linha_wi] = np.dot(input_slice.T, erros)

        return dW.transpose(2, 0, 1)
```

Gradiente em relação à entrada $\frac{\partial L}{\partial X} = \frac{\partial L}{\partial O}\frac{\partial O}{\partial X}$

$$\begin{bmatrix} \frac{\partial L}{\partial o_1} & \frac{\partial L}{\partial s_1} \\ \frac{\partial L}{\partial o_2} & \frac{\partial L}{\partial s_2} \\ \frac{\partial L}{\partial o_3} & \frac{\partial L}{\partial s_3} \\ \frac{\partial L}{\partial o_4} & \frac{\partial L}{\partial s_4} \\ \frac{\partial L}{\partial o_5} & \frac{\partial L}{\partial s_5} \end{bmatrix}$$

Quais erros da saída influenciaram cada elemento da entrada?

$$\frac{\partial L}{\partial o_1}\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \\ w_{3,1} & w_{3,2} & w_{3,3} \end{bmatrix} + \frac{\partial L}{\partial s_1}\begin{bmatrix} w`_{1,1} & w`_{1,2} & w`_{1,3} \\ w`_{2,1} & w`_{2,2} & w`_{2,3} \\ w`_{3,1} & w`_{3,2} & w`_{3,3} \end{bmatrix}$$

Possui a mesma forma do input já que é o gradiente em relação ao input.

$$\frac{\partial L}{\partial o_2}\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \\ w_{3,1} & w_{3,2} & w_{3,3} \end{bmatrix} + \frac{\partial L}{\partial s_2}\begin{bmatrix} w`_{1,1} & w`_{1,2} & w`_{1,3} \\ w`_{2,1} & w`_{2,2} & w`_{2,3} \\ w`_{3,1} & w`_{3,2} & w`_{3,3} \end{bmatrix}$$

```python
    def calcular_dX(self, erros:np.ndarray):

        dL_dX = np.zeros_like(self.saved_input) #erro em relação ao input
        qtd_linhas_erros = erros.shape[0]

        for i in range(qtd_linhas_erros):
            pos = i * self.stride

            #multiplicar cada elemento da linha da matriz de erros por cada
matriz de pesos que os gerou e somar tudo
            resultado = np.tensordot(erros[i], self.dilated_weights, axes=([0],
[0]))

            #somamos o resultado na região de onde veio o erro
            dL_dX[pos : pos + self.dilated_weights.shape[1]] += resultado


        if self.zero_padding == 0:
            return dL_dX
        return dL_dX[self.zero_padding : -self.zero_padding] #remover o padding
adicionado no forward

    def calcular_dB(self, erros):
        dB = np.sum(erros, axis=0) #soma dos erros já que dout/dB = 1
        return dB

    def backward(self, erros:np.ndarray, learning_rate:float = 0.001) ->
np.ndarray:
        dW = self.calcular_dW(erros)
        dB = self.calcular_dB(erros)
        dX = self.calcular_dX(erros)

        #otimizador padrão: gradient descent
        self.weights -= learning_rate * dW
        self.bias    -= learning_rate * dB
        return dX
```

Fully Connected

```python
class FullyConnected:
    def __init__(self, input_size:np.int64,output_size:np.int64,
weights:np.ndarray = None, bias:np.ndarray = None):
        if weights is None:
            weights = np.random.randn(output_size, input_size).astype(np.float32)
        else:
            assert weights.shape == (output_size, input_size), "Pesos não
compativeis com entrada e saida"

        if bias is None:
            bias = np.random.randn(output_size).astype(np.float32)
        else:
            assert bias.shape == (output_size,), "Bias não compativel com a
saida"

        self.weights = weights
        self.input_size = input_size
        self.output_size = output_size
        self.saved_input = None
        self.bias = bias

    def forward(self, input_data:np.ndarray) -> np.ndarray:
        self.saved_input = input_data

        return self.weights @ input_data + self.bias

    def backward(self, grad_output:np.ndarray, learning_rate:float = 0.001) -
>np.ndarray:
        dX = self.weights.T @ grad_output
        dW = grad_output @ self.saved_input.T
        dB = np.sum(grad_output, axis=1)

        #otimizador padrão: gradient descent
        self.weights -= learning_rate * dW
        self.bias    -= learning_rate * dB
        return  dX
```

Forward

$$Z = X \cdot W + B$$

Gradiente em relação ao peso

$$\frac{\partial L}{\partial W} = X^T \cdot \frac{\partial L}{\partial Z}$$

Gradiente em relação ao viés

$$\frac{\partial L}{\partial B} = \frac{\partial L}{\partial Z}$$

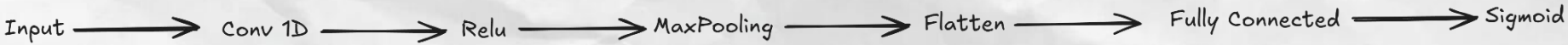
Gradiente em relação a entrada

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} \cdot W^T$$

Flatten


```python
class Flatten:
    def __init__(self):
        self.input_shape = None

    def forward(self, input_data):
        self.input_shape = input_data.shape #salvar o formato original
        # Formato de acesso: (L*C, 1)
        return input_data.T.flatten().reshape(-1, 1) #(formato vetor coluna)

    def backward(self, grad_output):
        #Desfaz  achatamento
        grad_transposed = grad_output.reshape(self.input_shape[1],
self.input_shape[0])
        return grad_transposed.T
```

Arquitetura



- **Fonte:** WISDM (*Wireless Sensor Data Mining*).
- **Sensores:** Acelerômetro (x,y,z)
- **Amostragem:** Recorte de 10 sujeitos.
    - Divisão: 8 para Treino / 2 para Teste.
- **Input:** Janelas temporais de **200 passos** de tempo.
- **Tarefa:** Classificação Binária (**Estático** vs. **Dinâmico**).

Configurações

Tabela 1: Arquitetura detalhada da CNN 1D utilizada no experimento

Camada	**Configuração**	**Detalhes**
Entrada	$Channels = 3$	Eixos X, Y, Z
Convolução 1D	$Filters = 4, Kernel = 3$	Sem padding, Stride=1
Ativação	ReLU	-
Pooling	MaxPool 1D	$Kernel = 2, Stride = 2$
Flatten	-	Vetorização de (4×99)
Fully Connected	$In = 396, Out = 1$	Neurônio único de saída
Saída	Sigmoid	Classificação Binária

Resultados dos experimentos